	detailed in Appendix A.5.		
Controlled components	Manage forms using controlled inputs.	Refactor uncontrolled forms to use controlled patterns.	Architect advanced form patterns using React Hooks and/or third-party libraries.
Suspense & concurrency	Use `React.lazy` and `Suspense` for code splitting.	Handle fallback UIs and lazy-loaded data efficiently.	Architect concurrent rendering strategies and implement streaming SSR (Server-Side Rendering).
Error Handling	Catch runtime errors in the UI using error boundaries. See appendix A.6 for details.	Implement class-based error boundaries for localized error recovery. See appendix A.6 for details.	Architect global error monitoring systems and design fallback UI strategies for robust applications.

With the fundamentals of React lifecycle and patterns in place, it is time to dive into practical interview questions.

Initial question (applied skill): "Can you spot a problem in listing 4.7?"

Listing 4.7 Example of Tooltip component

```
const Tooltip: React.FC<TooltipProps> = ({ text, ref }) => {
  const [position, setPosition] = useState<Position>({ top: 0, le
  useEffect(() => {
    if (ref.current) {
      const rect = ref.current.getBoundingClientRect();
      setPosition({
        top: rect.bottom + window.scrollY,
```